

AtPL: Sheet Three

REMINDER:

Proofs are hard.

Don't worry too much about them. All we want from you is to have a go. Best thing to do is time-box them so you don't waste time banging your head against the sheet. Your time is would be better spent understanding the solutions when they come out.

If stuck in a proof:

- Make sure you have broken into cases if its by induction
- Do the easier cases first
- Write down all the facts you know that are related to what you are trying to prove e.g. induction hypothesis or the rules
- Take a break.
- Set a timer - if you still have no idea by the time it goes off, just leave it, we will give you feedback on what you have down and you should ensure you understand the solution when it comes.

①

Here we go again with these trees. This is a step of the course, take time to understand it. The only difference between here and sheet one, is that the rules are more complex and plentiful.

(as there are so many rules I won't write them out - see slides for rules)

$$(i) \text{ plus(num}[i]; \text{num}[i]) \mapsto \text{num}[2]$$

I think of my rule selection as a pattern match here. I will want plus when applied to nums

$$\Rightarrow \text{D-PLUS} \frac{n_1 + n_2 = n}{\text{plus(num}[n_1]; \text{num}[n_2]) \mapsto \text{num}[n]}$$

$$\frac{1 + 1 = 2}{\text{plus(num}[1]; \text{num}[1]) \mapsto \text{num}[2]} \text{D-PLUS}$$

(ii)

$$\text{D-PLUS} \frac{1 + 1 = 2}{\text{plus(num}[1]; \text{num}[1]) \mapsto \text{num}[2]}$$

E-se-tm

$$\text{times}(\text{plus(num}[1]; \text{num}[1]); \text{num}[2]) \mapsto \text{times}(\text{num}[2]; \text{num}[2])$$

(iii)

$$\frac{\text{cat}(\text{str}[a]; \text{str}[b])}{\text{cat}(v; \text{str}[b])} \text{subst} \quad [\text{str}[a]/v]$$

$$\frac{}{\text{let}(\text{str}[a]; v. \text{cat}(v; \text{str}[b])) \mapsto \text{cat}(\text{str}[a]; \text{str}[b])} \text{D-LET}$$

$$\frac{}{\text{len}(\text{let}(\text{str}[a]; v. \text{cat}(v; \text{str}[b]))) \mapsto \text{len}(\text{cat}(\text{str}[a]; \text{str}[b]))} \text{D-LEN-1}$$

② Here we can pick which part of the program to reduce. To reduce we exchange the LHS of a rule for the RHS - the rule is, if you can justify the transition with a tree, we can transform.

i) $\text{times}(\text{plus}(\text{num}[1]; \text{num}[2]); \text{num}[2]) \mapsto^* \text{num}[4]$

$\text{times}(\text{plus}(\text{num}[1]; \text{num}[2]); \text{num}[2])$

→ justified by (i)

$\text{times}(\text{num}[2]; \text{num}[2])$

→

$\text{num}[4]$

ii)

Be careful to not bundle together single steps. What I do is think what I do individually reduce, and then double check a rule justifies it.

$\text{times}(\text{len}(\text{let}(\text{str}[a]; \text{v. cat}(v; \text{str}[b])))); \text{num}[2])$

→ justified by (iii)

$\text{times}(\text{len}(\text{cat}(\text{str}[a]; \text{str}[b]))); \text{num}[2])$

→

$\text{times}(\text{len}(\text{str}[ab])); \text{num}[2])$

→

$\text{times}(\text{num}[2]; \text{num}[2])$

→

$\text{num}[4]$

③ Same thing except at the end we go with a num/str. this is well typed.

(if we result in such a value).

Its a use case for these reductions too, showing you won't just do it (as a question says to).

(i) ~~let(str[a]; z.plus(len(z); len(z)))~~

→ ~~plus(len(z); len(z)) [str[a] / z]~~

= ~~plus(len(str[a]); len(str[a]))~~

→ ~~plus(num[1]; len(str[a]))~~

→ ~~plus(num[1]; num[1])~~

→ num[z] This reduces to a num which is a valid type ⇒ well typed

$$\frac{z \in \mathbb{N}}{\text{num}[z] \text{ val}} \text{ Val-Num}$$

ii) ~~let(len(str[a]); z.plus(z; z))~~

→ ~~let(num[1]; z.plus(z; z))~~

→ ~~plus(num[1]; num[1])~~

→ num[z] num ⇒ well typed

iii) ~~plus(let(len(str[a]); z.plus(z; z)); num[1])~~

→ * (ii)
~~plus(num[z]; num[1])~~

→ num[3] num ⇒ well typed

④ Existing rules:

$$\frac{e_1 \mapsto e'_1}{\text{plus}(e_1; e_2) \mapsto \text{plus}(e'_1; e_2)} \quad \text{D-PLUS-1}$$

$$\frac{e_1 \text{ val } e_2 \mapsto e'_2}{\text{plus}(e_1; e_2) \mapsto \text{plus}(e_1; e'_2)} \quad \text{D-PLUS-2}$$

this guy insists upon full reduction of e_1 before working on $\Rightarrow$ ~~reducing~~

New rules:

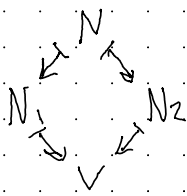
$$\frac{e_2 \mapsto e'_2}{\text{plus}(e_1; e_2) \mapsto \text{plus}(e_1; e'_2)} \quad \text{D-PLUS-1'}$$

$$\frac{e_2 \text{ val } e_1 \mapsto e'_1}{\text{plus}(e_1; e_2) \mapsto \text{plus}(e'_1; e_2)} \quad \text{D-PLUS-2'}$$

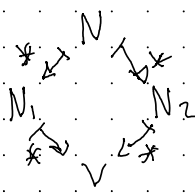
No this can not affect the final value. It doesn't matter which way you go.

those that did TLC, this is because $\mapsto$ satisfies the diamond property, meaning that its reflexive transitive closure $\mapsto^*$ satisfies Church-Rosser/confluence $\Rightarrow$ no matter what path, we will reach the same result.

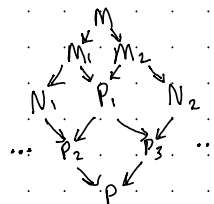
Diamond:



Confluence:



Proof by diagram chase:



you must draw diamonds for diamond prop

Now onto our proofs, which are ALL by induction. So let's remind ourselves of our recipe:

Proof by induction recipe:

1. Decide what to induct on.
 - In general, this is the most relevant judgement that you have all components for
 - which you have a choice between two of the same type, pick the one with the most interesting role in what you are trying to prove
2. Take the proof case by case; one case per rule
 - remember to assume your premises
 - write down the IH if it is a recursive rule

If you are stuck, write all the facts you know in front of you, and consider them. If any can help.

⑤ Step one: decide what to induct on

We are talking about vals, so induction on e val makes sense

Proof by induction on e val (always announce)

Step two: tackle proof case by case, one case per rule

Rules: (always good to remind ourselves)

$$\frac{n \in N}{\text{num}[n] \text{ val}} \text{ VAL-NUM}$$

$$\frac{s \in \Sigma^*}{\text{shr}[s] \text{ val}} \text{ VAL-STR}$$

(honestly writing the boring stuff helps me think about what next)

[Case: VAL-NUM]

$\Rightarrow e$ has shape $\text{num}[n]$

Ass(premise) = $n \in N$

Aim = $\vdash e : \text{Num}$ or $\vdash e : \text{shr}$

Our typing rule:

$$\frac{n \in N}{\vdash \text{num}[n] : \text{Num}} \text{ Num}$$

By Ass(premise) $\vdash e : \text{Num} \quad \square_{\text{VAL-NUM}}$

Notice how mechanical this is: I follow the recipe, write down relevant things and the proof falls out. All I need to do is connect the dots clearly and the reaper writing facts will help.

[Case: VAL-STR]

$\Rightarrow e$ has shape $\text{shr}(s)$

Ass(premise) = $s \in \Sigma^*$

Aim = $\vdash e : \text{Num}$ or $\vdash e : \text{shr}$

Typing rule:

$$\frac{s \in \Sigma^*}{\vdash \text{shr}[s] : \text{shr}} \text{ STR}$$

By Ass(premise) $\vdash \text{shr}[s] : \text{shr}$

oh look I have what I need to conclude

$\vdash \text{shr}[s] : \text{shr}$
surell $\checkmark$

$\square_{\text{VAL-STR}}$

$\square$

And that is all the cases $\checkmark$

⑥ Here the hint does step one for us to

Proof by induction on $e_1 \rightarrow^* e_2$

What is next?

Case-split!

Rules:

$$\frac{}{e \rightarrow^* e} \text{D-MULTI-REFL} \quad \text{zero steps is valid}$$

$$\frac{e \rightarrow e' \quad e' \rightarrow^* e''}{e \rightarrow^* e''} \text{D-MULTI-STEP}$$

[Case: D-MULTI-REFL]

Remember, to show something is admissible we must be able to produce a derivation of the conclusion from the premise.

GOAL: $e_1 \rightarrow^* e_3$

From the case we are in, we know the shape of the first premise is

$$e_1 \rightarrow e$$

$$\Rightarrow e_1 = e_2$$

GOAL': $e_2 \rightarrow^* e_3$

ASS (other premise) = $e_2 \rightarrow^* e_3$

Given that $e_1 = e_2$ from our case, and our other premise we can derive the conclusion $e_1 \rightarrow^* e_3$ showing this rule to be admissible in this case. $\square$ D-MULTI-REFL

[Case: D-MULTI-STEP]

$\Rightarrow$ the first premise has shape $e_1 \rightarrow^* e_2$ and derivation:

$$\frac{e_1 \rightarrow e' \quad e' \rightarrow^* e_2}{e_1 \rightarrow^* e_2}$$

Giving us: $e_1 \rightarrow e'$ (SHAPE 1)
 $e' \rightarrow^* e_2$ (SHAPE 2)

IH = the following rule is admissible:

$$\frac{e_1 \rightarrow^* e_{12} \quad e_{12} \rightarrow^* e_2}{e_1 \rightarrow^* e_2}$$

In other words, from $e_1 \rightarrow^* e_{12}$ and $e_{12} \rightarrow^* e_2$ we can conclude $e_1 \rightarrow^* e_2$ for some e_{12} .

ASS = we have a derivation for $e_2 \rightarrow^* e_3$ (ASS)

GOAL: $e_1 \rightarrow^* e_3$

SHAPE 1: $e_1 \rightarrow e'$
Applying IH to SHAPE 2 and ASS to give us $e' \rightarrow^* e_3$

$$\frac{e_1 \rightarrow e' \quad e' \rightarrow^* e_3}{e_1 \rightarrow^* e_3} \text{D-MULTI-STEP}$$

Specialised IH

$$\frac{e' \rightarrow^* e_2 \quad e_2 \rightarrow^* e_3}{e' \rightarrow^* e_3}$$

⑦

Preservation: If $\vdash e : \tau$ and $e \mapsto e'$ then $\vdash e' : \tau$

Proof by induction on $e \mapsto e'$.

[Case: D-PLUS]

$$\frac{n_1 + n_2 = n}{\text{plus}(\text{Num}[n_1]; \text{Num}[n_2]) \mapsto \text{Num}[n]} \quad \text{D-PLUS}$$

ASS1 = $n_1 + n_2 = n$ (premise)

ASS2 = $\vdash e : \tau$ (antecedent)

GOAL = $\vdash e' : \tau$

In this case $e' = \text{Num}[n]$

$\Rightarrow$ we can use the Num rule

$$\frac{n \in \mathbb{N}}{\vdash \text{Num}[n] : \text{Num}} \quad \text{Num} \quad \square \text{D-PLUS}$$

[Case: D-PLUS-1]

$$\frac{e_1 \mapsto e'_1}{\text{plus}(e_1; e_2) \mapsto \text{plus}(e'_1; e_2)} \quad \text{D-PLUS-1}$$

IH = if $\vdash e_1 : \tau$ and $e_1 \mapsto e'_1$ then $\vdash e'_1 : \tau$

ASS1 = $\text{plus}(e_1; e_2) : \tau$ (antecedent)

To unlock our IH, we need the type of e_1 , luckily we can use inversion to tell us that.

From inversion, we know $e_1 : \text{Num}$ (INV1)
and $e_2 : \text{Num}$ (INV2)

GOAL = $\text{plus}(e'_1; e_2) : \tau$

In this case we know $\tau = \text{Num}$

GOAL' = $\text{plus}(e'_1; e_2) : \text{Num}$

We can provide the IH with INV1 to get $e'_1 : \text{Num}$ (IH)
that is all the facts I can think of, so let's try and derive our goal and hope we have the required premises:

$$\frac{\begin{array}{c} : \text{IH} \\ \vdash e'_1 : \text{Num} \end{array} \quad \begin{array}{c} : \text{INV2} \\ \vdash e_2 : \text{Num} \end{array}}{\vdash \text{plus}(e'_1; e_2) : \text{Num}} \quad \text{Plus}$$

[Case: D-LET]

$\square \text{D-PLUS-1}$

$$\frac{}{\text{let}(e_1; x. e_2) \mapsto e_2[e_1/x]} \quad \text{D-LET}$$

ASS1 = $\vdash \text{let}(e_1; x. e_2) : \tau$

GOAL = $e_2[e_1/x]$

Since we want to learn about the type of e_1 , let's turn to inversion.

Inversion tells us $\exists \sigma \text{ s.t. } \vdash e_1 : \sigma \text{ and } x : \sigma \vdash e_2 : \tau$

This is just the substitution lemma:

If $V \vdash e : \tau$ and $V, x : \tau \vdash u : \sigma$
then $V \vdash u[e/x] : \sigma$

where we specialise V to empty, e to e_1 , etc. $\square \text{D-LET}$

The rest of the cases are just remixes of these.

⑧ if $\vdash e:\tau$ then either eval
or $e \mapsto e'$ for some e'

Following the theme, this is a proof by induction, so again we will use our recipe.

What is step one?

Decide who we will induct on!

Since we want this to be the case for all $\vdash e:\tau$, makes sense to induct on that.

Proof by induction on $\vdash e:\tau$.

You should know the drill by now, what is next?

Case split!

[Case: Var]

$$\frac{}{\vdash, x:\sigma \vdash x:\sigma} \text{VAR}$$

$\Rightarrow$ our context is non-empty.

This means our antecedent ($\vdash e:\tau$) is not empty, so we are done.

$\square_{\text{VAR}}$

[Case: Num]

$$\frac{n \in \mathbb{N}}{\vdash \text{num}[n] : \text{Num}} \text{NUM}$$

ASS = $n \in \mathbb{N}$

GOAL = $e \text{ val}$ or $e \mapsto e'$

Since we have the premise (ASS) to show $\text{num}[n] \text{ val}$, let's do that.

$$\frac{\frac{}{n \in \mathbb{N}} \text{ASS}}{\text{num}[n] \text{ val}} \text{Num}$$

[Case: STR]

Num, but stringified.

[Case: PLUS]

$$\frac{\vdash e_1 : \text{Num} \quad \vdash e_2 : \text{Num}}{\vdash \text{plus}(e_1, e_2) : \text{Num}} \text{PLUS}$$

IH1 = either $e_1 \text{ val}$ or $e_1 \mapsto e'$ for some e'

IH2 = either $e_2 \text{ val}$ or $e_2 \mapsto e'$ for some e'

GOAL = either $\vdash \text{plus}(e_1, e_2) : \tau$ or $\text{plus}(e_1, e_2) \mapsto e'$ for some e'

Focussing on IH1

[Subcase: $e_1 \text{ val}$]

[Subsubcase: $e_2 \text{ val}$]

So they are both values and they are being added, in which case this checks out that they are both nums. Luckily, there is a lemma that says exactly that.

By the canonical forms lemma:

1. If $\tau = \text{num}$ then $e = \text{num}[n]$ for some n

2. If $\tau = \text{str}$ then $e = \text{str}[s]$ for some s

$e_1 = \text{num}[n_1]$ and $e_2 = \text{num}[n_2]$ for some $n_1, n_2 \in \mathbb{N}$

In other words, in this case

$e = \text{plus}(\text{num}[n_1], \text{num}[n_2])$

Happily this is the LHS of an axiom for case $\mapsto$, so we can prove that side of the goal.

$$\text{D-PLUS} \quad \frac{}{e = \text{plus}(\text{num}[n_1], \text{num}[n_2]) \mapsto \text{num}[n_1 + n_2]}$$

$\Rightarrow \exists e' \text{ s.t. } e \mapsto e'$, specifically

$e' = \text{num}[n_1 + n_2]$

$\square_{\text{PLUS } e_2 \text{ val subcase}}$

[Subsubcase: $\exists e' \text{ s.t. } e_2 \mapsto e'$]

Reminder we have:

$e_1 \text{ val}$

ASS1

$\exists e' \text{ s.t. } e_2 \mapsto e'$

ASS2

and we wanna make a value $e' \mapsto$ thing

Since we have a $\mapsto$ thing already, we don't gonna make a value.

BUT what we have fits the requirement for D-PLUS-2

$$\frac{\frac{}{e_1 \text{ val}} \text{ASS1} \quad \frac{}{e_2 \mapsto e'} \text{ASS2}}{\text{plus}(e_1, e_2) \mapsto \text{plus}(e_1, e')} \text{D-PLUS-2}$$

$\Rightarrow \exists$ some term namely $\text{plus}(e_1, e')$.

$\square_{\text{PLUS subcase } e_1 \mapsto e'}$

[Subcase: $\exists e' \text{ s.t. } e_1 \mapsto e'$]

ASS = $\exists e' \text{ s.t. } e_1 \mapsto e'$

Looking at our rules again, the case we are in gives us the premise we need to make a $\mapsto$ thing:

$$\frac{}{e_1 \mapsto e'} \text{ASS}$$

$$\frac{}{\text{plus}(e_1, e_2) \mapsto \text{plus}(e', e_2)} \text{D-PLUS-1}$$

$\square_{\text{PLUS subcase } \exists e' \text{ s.t. } e_1 \mapsto e'}$

$\square_{\text{PLUS}}$

[Case: LET]

$$\frac{\vdash e_1 : \sigma_1 \quad \vdash, x:\sigma_1 \vdash e_2 : \sigma_2}{\vdash \text{let}(e_1, x.e_2) : \sigma_2} \text{LET}$$

Happily this is an easy one, as let terms always reduce with the D-LET axiom.

$$\frac{}{\text{let}(e_1, x.e_2) \mapsto e_2[e_1/x]} \text{D-LET}$$

$\square_{\text{D-LET}}$

The rest are like PLUS.

$\square$

① I've got nothing to add to the actual answers. - just goes to show how important steps are. is to think carefully about what your induction - could make your life easier.